1. Difference between Local, Instance, and Static Variables

Feature	Local Variable	Instance Variable	Static Variable
Declared	Inside a method, constructor, or block	Inside a class but outside methods	Inside a class with the `static` keyword
Ownership	Belongs to a method	Belongs to an object	Belongs to the class
Memory Created	When the method is called	When an object is created	When the class is loaded
Number of Copies	One per method call	One per object	Only one shared copy
Default Value	No default value	Gets default values (`0`, `null`, `false`, etc.)	Gets default values
Access	Only inside the method	Through an object	Through the class name or object

Example

```
class Student {

      int age;              // Instance variable
      static String school = "ABC"; // Static variable

      void display() {
      int marks = 90;   // Local variable
        System.out.println(age);
        System.out.println(school);
      System.out.println(marks);
      }
}
```

---

2. Why must local variables be initialized before use?

Local variables **do not receive default values** from Java.

This is done to prevent using random or garbage values.

Incorrect

```
public class Test {
      public static void main(String[] args) {
      int x;
        System.out.println(x);   // Compile-time error
    }
}
```

Error:

```
variable x might not have been initialized
```

Correct

```
int x = 10;
System.out.println(x);
```

Why instance/static variables don't need initialization

```
class Test {
      int a;       // 0
      boolean b;   // false
      String s;    // null
}
```

The JVM automatically initializes them with default values.

---

3. Difference between `final`, `static`, and `static final`

Keyword	Meaning
`final`	Cannot be reassigned after initialization
`static`	Belongs to the class instead of objects
`static final`	Class-level constant (shared and cannot be changed)

`final`

```
final int x = 10;

x = 20;   // Error
```

`static`

```
class Test {
      static int count = 0;
}
```

All objects share the same `count`.

`static final`

```
class MathUtil {
      static final double PI = 3.14159;
}
```

Used for constants.

Convention:

```
public static final int MAX_SIZE = 100;
```

---

4. Where are local variables and instance variables stored in memory?

Local Variables

Stored in the **stack memory** (specifically, in the stack frame of the executing method).

```
void fun() {
      int x = 10;
}
```

When `fun()` finishes, `x` is automatically removed.

Instance Variables

Stored in the **heap memory** as part of the object.

```
class Student {
      int age;
}

Student s = new Student();
```

The object (including `age`) lives in the heap until it becomes unreachable and is eligible for garbage collection.

Static Variables

Stored in the **method area (class metadata area)**, which is shared across all instances of the class.

Memory Summary

```
Stack Memory
-------------
main()
x
y
method()

Heap Memory
------------
Student object
 age
 name

Method Area
```

```
------------
Class information
static variables
```

---

5. Can we change the value of a `final` reference variable?

Yes, the object's state can change, but the reference itself cannot point to a different object.

Example

```
class Student {
	int age;
}

public class Test {
	public static void main(String[] args) {

	final Student s = new Student();

	s.age = 20;		// Allowed

	// s = new Student(); // Compile-time error
	}
}
```

Explanation

`final` makes the **reference** immutable, not the object.

Think of it like this:

```
final Student s
	|
	V
	Student Object
```

You cannot make `s` point somewhere else:

```
s = new Student();   // Not allowed
```

But you can modify the object that `s` refers to:

```
s.age = 25;   // Allowed
```

If the object itself is designed to be immutable (for example, a `String`), then its state also cannot be changed.

Quick Revision

- **Local variable**: Declared inside methods, stored on the stack, no default value, must be initialized before use.

- **Instance variable**: Belongs to each object, stored in the heap, receives default values.
- **Static variable**: Belongs to the class, shared by all objects, stored in the method area.
- **`final`**: Prevents reassignment after initialization.
- **`static`**: Makes a member belong to the class rather than individual objects.
- **`static final`**: Creates a shared constant whose value cannot be changed.
- **`final` reference variable**: The reference cannot be changed to point to another object, but the object's fields can still be modified unless the object itself is immutable.

---

1. Difference between Primitive and Wrapper Types

Java has **primitive data types** and **wrapper classes**.

Primitive Type	Wrapper Class
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

Primitive Types

- Store actual values.
- Faster because they don't create objects.
- Stored directly in memory.
- Cannot call methods.

int x = 10;

Wrapper Types

- Objects that wrap primitive values.
- Stored as objects on the heap.
- Provide useful methods.

Integer x = 10;

System.out.println(x.toString());
System.out.println(x.compareTo(20));

Comparison

Primitive	Wrapper
Not an object	Object
Faster	Slightly slower
Less memory	More memory
Cannot be `null`	Can be `null`
No methods	Has many utility methods

---

2. Why can't collections store primitive types directly?

Collections like `ArrayList`, `HashMap`, and `HashSet` store **objects**, not primitive values.

For example:

ArrayList<int> list = new ArrayList<>(); // Error

This is invalid because `int` is **not an object**.

Instead, use wrapper classes:

ArrayList<Integer> list = new ArrayList<>();

list.add(10);
list.add(20);

The compiler automatically converts (`autoboxes`) the `int` values into `Integer` objects.

Why Java does this:

- Collections use **generics**, and generics work only with reference types (objects).
- Primitive types are not reference types.

---

3. What is Autoboxing and Unboxing?

Autoboxing

Automatic conversion of a primitive into its wrapper object.

Integer num = 10;

Compiler converts it to:

Integer num = Integer.valueOf(10);

Unboxing

Automatic conversion of a wrapper object into a primitive.

Integer num = 20;

int x = num;

Compiler converts it to:

int x = num.intValue();

Example

ArrayList<Integer> list = new ArrayList<>();

```
list.add(100);   // Autoboxing

int value = list.get(0);   // Unboxing

System.out.println(value);
```

---

4. What are the default values of `int`, `boolean`, and object references?

These default values apply **only to instance and static variables**, not local variables.

Type	Default Value
`int`	`0`
`boolean`	`false`
Object reference	`null`

Example

```
class Student {

        int age;
        boolean passed;
        String name;

        void display() {
        System.out.println(age);        // 0
        System.out.println(passed);   // false
        System.out.println(name);      // null
        }
}
```

Local variables do **not** get default values.

```
void test() {
        int x;
        System.out.println(x); // Compile-time error
```

}

---

5. Why should `BigDecimal` be used for currency instead of `double`?

`double` uses **binary floating-point representation**, which cannot exactly represent many decimal fractions. This leads to small rounding errors.

Example with `double`

double price = 0.1 + 0.2;

System.out.println(price);

Output:

0.30000000000000004

This is unacceptable for financial calculations.

Using `BigDecimal`

import java.math.BigDecimal;

BigDecimal price1 = new BigDecimal("0.1");
BigDecimal price2 = new BigDecimal("0.2");

BigDecimal total = price1.add(price2);

System.out.println(total);

Output:

0.3

The result is exact because `BigDecimal` stores decimal values with arbitrary precision instead of using binary floating-point.

Why `BigDecimal` is preferred for money

- Exact decimal arithmetic (no floating-point rounding errors).
- High precision for financial calculations.
- Supports configurable rounding modes.
- Widely used in banking, accounting, taxation, and e-commerce.

Comparison

`double`	`BigDecimal`
Fast	Slower
Floating-point, approximate	Decimal, exact
May introduce rounding errors	No unexpected rounding errors
Good for scientific calculations	Best for financial calculations

---

Quick Revision

- **Primitive types** store raw values, while **wrapper types** are objects that encapsulate primitive values.
- **Collections** store only objects, so wrapper classes (e.g., `Integer`) are used instead of primitives (e.g., `int`).
- **Autoboxing** automatically converts a primitive to its wrapper; **unboxing** converts the wrapper back to a primitive.
- Default values for instance/static variables:
  - `int` → `0`
  - `boolean` → `false`
  - Object reference → `null`
- Use **`BigDecimal`** for currency because it provides precise decimal arithmetic and avoids the rounding errors inherent in `double`.

Here are the Java concepts explained with examples.

---

1. Difference between `==` and `.equals()`

The main difference is:

- `==` compares **references** (or values for primitives).
- `.equals()` compares **contents** (if the class overrides it appropriately).

For Primitive Types

`==` compares actual values.

```
int a = 10;
int b = 10;

System.out.println(a == b);
```

Output:

```
true
```

For Objects

```
String s1 = new String("Java");
String s2 = new String("Java");

System.out.println(s1 == s2);
System.out.println(s1.equals(s2));
```

Output:

```
false
true
```

Explanation:

- `==` checks whether `s1` and `s2` refer to the **same object**.
- `.equals()` checks whether they contain the **same text**.

Memory Illustration

s1 -----> "Java" (Object 1)

s2 -----> "Java" (Object 2)

Different objects → `==` is `false`.

Same content → `.equals()` is `true`.

Summary

`==`	`.equals()`
Compares references (objects) or values (primitives)	Compares object contents (if implemented)
Can compare primitives	Cannot be used with primitives
Faster	Usually slightly slower because it compares contents

---

2. Difference between `&&` and `&`

Both can be used with boolean values, but they behave differently.

`&&` (Logical AND)

- Short-circuit operator.
- Stops evaluating when the left operand is `false`.

boolean result = false && true;

System.out.println(result);

Output:

false

The second operand isn't evaluated because the result is already known.

`&` (Bitwise AND / Non-Short-Circuit AND)

When used with booleans:

- Evaluates **both** operands every time.

boolean result = false & true;

System.out.println(result);

Output:

false

Even though the result is `false`, the second operand is still evaluated.

Example

int x = 5;

if (false && (++x > 5)) {
}

System.out.println(x);

Output:

5

Because `++x` never executes.

Now with `&`:

int x = 5;

if (false & (++x > 5)) {
}

System.out.println(x);

Output:

6

The second expression is evaluated.

Summary

`&&`	`&`
Logical AND	Bitwise AND (integers) / Non-short-circuit logical AND (booleans)
Stops early if left side is `false`	Always evaluates both sides

---

3. What is Short-Circuit Evaluation?

Short-circuit evaluation means Java **skips evaluating the second operand when the result is already determined**.

Applies to:

- `&&`
- `||`

With `&&`

false && anything

Result is always `false`, so Java skips `anything`.

Example:

```
int x = 10;

if (x < 5 && ++x > 0) {
}

System.out.println(x);
```

Output:

10

`++x` never runs.

With `||`

true || anything

Result is already `true`, so Java skips the second operand.

Example:

```
int x = 10;

if (true || ++x > 0) {
}

System.out.println(x);
```

Output:

10

Benefits

- Improves performance by avoiding unnecessary evaluation.
- Helps prevent exceptions.

Example:

```
String s = null;

if (s != null && s.length() > 0) {
}
```

If `s` is `null`, `s.length()` is never called, avoiding a `NullPointerException`.

---

4. What is Operator Precedence?

Operator precedence determines **which operator is evaluated first** in an expression.

Example:

int result = 5 + 3 * 2;

Multiplication has higher precedence than addition.

Equivalent to:

5 + (3 * 2)

Output:

11

Common Operator Precedence (Highest → Lowest)

Operator	Meaning
`()`	Parentheses
`++ --`	Increment/Decrement
`* / %`	Multiplication, Division, Modulus
`+ -`	Addition, Subtraction
`< > <= >=`	Relational
`== !=`	Equality
`&&`	Logical AND
`		
`=`	Assignment

Example

int x = 5 + 2 * 3;

Result:

11

Using parentheses:

int x = (5 + 2) * 3;

Result:

21

---

5. Explain Pre-Increment and Post-Increment

Java provides two increment operators.

Pre-Increment (`++x`)

- Increment first.
- Use the new value.

```
int x = 5;

int y = ++x;

System.out.println(x);
System.out.println(y);
```

Output:

6
6

Steps:

x = 5
++x → x becomes 6
y = 6

Post-Increment (`x++`)

- Use the current value.

- Increment afterward.

```
int x = 5;

int y = x++;

System.out.println(x);
System.out.println(y);
```

Output:

```
6
5
```

Steps:

```
x = 5
y = 5
x becomes 6
```

Comparison

Pre-Increment (++x)	Post-Increment (x++)
Increment first	Use current value first
Then return value	Increment afterward

Example

```
int a = 10;

System.out.println(++a);
System.out.println(a++);
System.out.println(a);
```

Output:

```
11
11
12
```

Explanation:

1. `++a` → `a` becomes `11`, prints `11`.
2. `a++` → prints `11`, then `a` becomes `12`.
3. Prints `12`.

Quick Revision

- `==` compares primitive values or object references; `.equals()` compares object contents (when implemented).
- `&&` is a short-circuit logical AND; `&` always evaluates both operands (and also acts as a bitwise AND for integer types).
- **Short-circuit evaluation** skips unnecessary evaluation with `&&` and `||`, improving performance and helping avoid errors like `NullPointerException`.
- **Operator precedence** defines the order in which operators are evaluated; parentheses `()` override the default order.
- **Pre-increment (`++x`)** increments before the value is used, while **post-increment (`x++`)** uses the current value first and increments afterward.

---

1. What is the difference between a Class and an Object?

A **class** is a blueprint or template, while an **object** is a real instance created from that blueprint.

Class

- Defines properties (fields) and behaviors (methods).
- Does not occupy memory for instance variables until an object is created.

```
class Student {
    String name;
    int age;
```

```
    void study() {
    System.out.println(name + " is studying");
    }
}
```

Object

An object is created using the `new` keyword.

```
Student s1 = new Student();

s1.name = "Alice";
s1.age = 20;

s1.study();
```

Output

```
Alice is studying
```

Real-Life Example

Class	Object
Car	My Honda City
Student	Alice
Mobile	My iPhone
Employee	John

Think of it this way:

- **Class** = Building blueprint
- **Object** = Actual building

Comparison

Class	Object

Blueprint	Instance of a class
Logical entity	Physical entity in memory
Created using `class` keyword	Created using `new` keyword
No instance memory allocated	Occupies heap memory

---

2. How do you decide what should become a class in a real project?

A class should represent a **real-world entity or concept** that has:

- **Attributes (state)**
- **Behaviors (actions)**

Ask yourself:

"Does this thing have its own data and operations?"

If yes, it is a good candidate for a class.

Example: Library Management System

Possible classes:

Book
Member
Librarian
Author
Library
Transaction

Example:

class Book {

```
    String title;
    String author;
    double price;

    void issueBook() {
    }

    void returnBook() {
    }
}
```

Example: Banking Application

Classes:

Customer
Account
Transaction
Bank
Loan
Card

```
class Account {

    int accountNumber;
    double balance;

    void deposit() {
    }

    void withdraw() {
    }
}
```

What should NOT become a class?

Simple values usually remain variables.

Examples:

```
String city;
int age;
double salary;
```

You don't normally create separate classes for these unless they need their own behavior.

Rule of Thumb

If it has:

- Identity
- Data
- Behavior

→ Make it a class.

---

3. What is Object State and Behavior?

Every object has:

- **State**
- **Behavior**

State

State is the **data stored inside an object**.

Represented by fields (instance variables).

```
class Car {

    String color;
    int speed;
}
```

For a car:

Color = Red
Speed = 80

These values represent its **state**.

Behavior

Behavior is **what the object can do**.

Represented by methods.

```
class Car {

        void start() {
        }

        void stop() {
        }

        void accelerate() {
        }
}
```

Example

```
class Dog {

        String name;
        int age;

        void bark() {
        System.out.println("Woof!");
        }
}
```

Object:

```
Dog d = new Dog();

d.name = "Tommy";
d.age = 4;

d.bark();
```

State:

name = Tommy
age = 4

Behavior:

bark()

---

4. Can a Java class exist without creating objects?

Yes.

A class can exist without creating any objects.

Example:

```
class Student {

	int age;

	void study() {
	}
}
```

This class compiles successfully even if no object is ever created.

Static Members

A class with only static members doesn't require objects.

```
class MathUtil {

	static int square(int x) {
	return x * x;
	}
}
```

Usage:

```
System.out.println(MathUtil.square(5));
```

Output:

25

No object is needed because `square()` is static.

When are objects required?

Objects are required to access **instance members**.

```
class Student {

        String name;
}

Student s = new Student();
```

Without an object, `name` cannot be accessed.

---

5. What is the difference between POJO, DTO, and Entity?

These terms describe different roles a Java class can play.

Feature	POJO	DTO	Entity
Full Form	Plain Old Java Object	Data Transfer Object	Persistent Object
Purpose	General Java object	Transfer data	Represent a database table
Contains Business Logic	Can	Usually no	Sometimes
Database Mapping	No	No	Yes
Used In	Any layer	API/service communication	Persistence layer

---

POJO (Plain Old Java Object)

A simple Java class.

```
class Student {

        private String name;
        private int age;

        // Constructors

        // Getters

        // Setters
}
```

Characteristics:

- No special inheritance requirement.
- No framework dependency.
- Simple Java object.

DTO (Data Transfer Object)

Used to transfer data between layers.

Example:

```
class StudentDTO {

        private String name;
        private int age;

        // Getters and setters
}
```

Purpose:

```
Database
        ↓
Entity
```

↓
DTO
↓
Client

DTOs often exclude sensitive fields.

Example:

Entity:

id
name
password
salary

DTO:

name

Only the required data is sent.

Entity

Represents a database table.

Example (using JPA annotations):

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
class Student {

    @Id
    private int id;

    private String name;
}
```

Each object corresponds to a row in the database.

Example table:

id	name
1	Alice
2	Bob

Real Project Flow

Database
↑↓
Entity
↓
Service
↓
DTO
↓
REST API
↓
Frontend

- **Entity** interacts with the database.
- **DTO** transfers only the required data between layers.
- **POJO** is a general term for any plain Java object; many DTOs and Entities are also POJOs because they are ordinary Java classes.

Quick Revision

- **Class**: Blueprint that defines fields and methods.
- **Object**: Instance of a class created with `new`.
- **Object state**: Values of an object's fields.
- **Object behavior**: Actions defined by its methods.
- **A class can exist without objects**, especially if it contains only static members or is simply a type definition.
- **POJO**: A simple Java object with no special framework requirements.

- **DTO**: Used to transfer data between application layers or systems.
- **Entity**: A class mapped to a database table, typically using JPA/Hibernate annotations.

---

1. What is Inheritance?

Inheritance is an Object-Oriented Programming (OOP) feature where one class acquires the properties (fields) and behaviors (methods) of another class.

- The existing class is called the **parent (superclass)**.
- The new class is called the **child (subclass)**.

Java uses the `extends` keyword.

Example

```
class Animal {

        void eat() {
        System.out.println("Animal is eating");
        }
}

class Dog extends Animal {

        void bark() {
      System.out.println("Dog is barking");
        }
}

public class Main {
        public static void main(String[] args) {

        Dog d = new Dog();

           d.eat();        // Inherited method
        d.bark();   // Dog's own method
        }
```

}

Output

Animal is eating
Dog is barking

Benefits

- Code reusability
- Easy maintenance
- Supports polymorphism
- Reduces duplicate code

---

2. Difference between is-a and has-a Relationship

Is-a Relationship (Inheritance)

An object **is a type of** another object.

Uses `extends`.

Example

```
class Animal {
}

class Dog extends Animal {
}
```

Here:

Dog is an Animal

Examples:

- Dog **is an** Animal

- Car **is a** Vehicle
- Student **is a** Person

Has-a Relationship (Composition)

One object **contains** another object.

Example

```
class Engine {

        void start() {
        System.out.println("Engine Started");
        }
}

class Car {

        private Engine engine = new Engine();

        void drive() {
        engine.start();
        System.out.println("Car is moving");
        }
}
```

Output:

```
Engine Started
Car is moving
```

Here:

Car has an Engine

Comparison

Is-a	Has-a
Inheritance	Composition
Uses `extends`	Uses object references

Represents specialization	Represents containment
Example: Dog is an Animal	Example: Car has an Engine

---

3. Why does Java not support multiple class inheritance?

Java **does not allow a class to extend multiple classes** because it can lead to ambiguity, commonly known as the **Diamond Problem**.

Example (Hypothetical)

```
class A {
	void display() {
	System.out.println("A");
	}
}

class B extends A {
	void display() {
	System.out.println("B");
	}
}

class C extends A {
	void display() {
	System.out.println("C");
	}
}

// Not allowed in Java
class D extends B, C {
}
```

If `D` calls:

```
display();
```

Which version should execute?

- `B.display()`?
- `C.display()`?

Java avoids this ambiguity by allowing **only one parent class**.

How does Java achieve multiple inheritance?

Through **interfaces**.

```
interface Camera {
	void click();
}

interface MusicPlayer {
	void playMusic();
}

class Mobile implements Camera, MusicPlayer {

	public void click() {
	System.out.println("Photo clicked");
	}

	public void playMusic() {
	System.out.println("Playing music");
	}
}
```

A class can implement multiple interfaces.

---

4. What is Method Overriding?

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Parent Class

```
class Animal {
```

```
    void sound() {
    System.out.println("Animal makes sound");
    }
}
```

Child Class

```
class Dog extends Animal {

    @Override
    void sound() {
    System.out.println("Dog barks");
    }
}
```

Test

```
public class Main {

    public static void main(String[] args) {

    Dog d = new Dog();
    d.sound();
    }
}
```

Output

```
Dog barks
```

Rules for Overriding

- Same method name.
- Same parameter list.
- Compatible return type (same or covariant).
- Child method cannot reduce visibility.
- `static`, `final`, and `private` methods cannot be overridden.

---

5. When should inheritance be avoided?

Inheritance is powerful, but it should **not** be used just to reuse code.

Avoid inheritance when:

1. There is no true "is-a" relationship

Bad example:

```
class Printer {
}

class Employee extends Printer {
}
```

An employee is **not** a printer.

---

2. Behavior may change independently

Example:

Car has an Engine

The engine can change without changing what a car is.

Composition is better.

---

3. You want loose coupling

Inheritance creates **tight coupling**.

Changes in the parent class can affect all child classes.

---

4. Only code reuse is needed

If you're inheriting solely to reuse methods, composition is often the better choice.

Rule

Use inheritance only when there is a genuine **is-a** relationship.

---

6. Difference between Inheritance and Composition

Inheritance

Uses an **is-a** relationship.

```
class Animal {

        void eat() {
      System.out.println("Eating");
        }
}

class Dog extends Animal {

        void bark() {
        System.out.println("Barking");
        }
}
```

Dog is an Animal

Composition

Uses a **has-a** relationship.

```
class Engine {

        void start() {
        System.out.println("Engine Started");
        }
}

class Car {
```

```
    private Engine engine = new Engine();

    void drive() {
    engine.start();
    }
}
```

Car has an Engine

Comparison

Inheritance	Composition
Is-a relationship	**Has-a** relationship
Uses `extends`	Uses object references
Tight coupling	Loose coupling
Parent changes can affect children	Components can change independently
Less flexible	More flexible
Best for specialization	Best for assembling functionality

Which is better?

Modern object-oriented design generally favors:

Composition over Inheritance

because it produces code that is:

- More flexible
- Easier to test
- Easier to maintain
- Less tightly coupled

However, inheritance is still the right choice when there is a clear and permanent **is-a** relationship (for example, `Dog` extends `Animal`).

Quick Revision

- **Inheritance** allows a child class to reuse and specialize the properties and methods of a parent class.
- **Is-a** → Inheritance (`Dog` is an `Animal`).
- **Has-a** → Composition (`Car` has an `Engine`).
- Java doesn't support **multiple class inheritance** to avoid ambiguity (the Diamond Problem); it uses **interfaces** for multiple inheritance of type.
- **Method overriding** lets a child class provide its own implementation of an inherited method.
- Avoid inheritance when there is no real **is-a** relationship, when you only want code reuse, or when loose coupling is more important.
- **Composition** is generally preferred over inheritance because it provides greater flexibility and maintainability.

---

1. What is Polymorphism?

Polymorphism means **"many forms."**

In Java, polymorphism allows the **same method call** to behave differently depending on the object that invokes it.

There are two types:

1. **Compile-time Polymorphism (Method Overloading)**
2. **Run-time Polymorphism (Method Overriding)**

Example of Runtime Polymorphism

class Animal {

```
	void sound() {
	System.out.println("Animal makes sound");
	}
}

class Dog extends Animal {

	@Override
	void sound() {
	System.out.println("Dog barks");
	}
}

class Cat extends Animal {

	@Override
	void sound() {
	System.out.println("Cat meows");
	}
}

public class Main {

	public static void main(String[] args) {

	Animal a;

	    a = new Dog();
	a.sound();

	a = new Cat();
	a.sound();
	}
}
```

Output

```
Dog barks
Cat meows
```

Notice that the same method call:

```
a.sound();
```

produces different behavior depending on the actual object.

---

2. Difference between Overloading and Overriding

Method Overloading

Two or more methods have:

- Same name
- Different parameter lists

Occurs **within the same class** (or inherited class).

Example

```
class Calculator {

    int add(int a, int b) {
    return a + b;
    }

    int add(int a, int b, int c) {
    return a + b + c;
    }

    double add(double a, double b) {
    return a + b;
    }
}
```

Compiler decides which method to call.

Method Overriding

A child class provides a new implementation of a parent class method.

Example

```
class Animal {

	void sound() {
	System.out.println("Animal sound");
	}
}

class Dog extends Animal {

	@Override
	void sound() {
	System.out.println("Dog barks");
	}
}
```

The JVM decides at runtime which implementation to execute.

Comparison

Overloading	Overriding
Same class (or inherited scope)	Parent and child classes
Same method name	Same method name
Different parameters	Same parameters
Compile-time polymorphism	Runtime polymorphism
Compiler resolves the call	JVM resolves the call at runtime

---

3. Can static methods be overridden?

No.

Static methods belong to the **class**, not to objects.

If a child class declares a static method with the same signature, it **hides** the parent's static method rather than overriding it.

Example

```
class Parent {

	static void show() {
	System.out.println("Parent");
	}
}

class Child extends Parent {

	static void show() {
	System.out.println("Child");
	}
}
Parent p = new Child();

p.show();
```

Output

Parent

Reason:

Static methods are resolved using the **reference type**, not the actual object.

This is called **method hiding**, not overriding.

---

4. Can private methods be overridden?

No.

Private methods are **not inherited**, so child classes cannot override them.

Example

```
class Parent {

	private void display() {
	System.out.println("Parent");
	}
}
```

```
class Child extends Parent {

	void display() {
	System.out.println("Child");
	}
}
```

This is **not overriding**.

The child simply defines a new method with the same name.

---

5. What is Dynamic Method Dispatch?

Dynamic Method Dispatch is the mechanism by which Java decides **at runtime** which overridden method to execute.

It is the foundation of **runtime polymorphism**.

Example

```
class Animal {

	void sound() {
	System.out.println("Animal");
	}
}

class Dog extends Animal {

	@Override
	void sound() {
	System.out.println("Dog");
	}
}

class Cat extends Animal {

	@Override
	void sound() {
	System.out.println("Cat");
	}
```

```
}
Animal animal;

animal = new Dog();
animal.sound();

animal = new Cat();
animal.sound();
```

Output

```
Dog
Cat
```

What happens?

Reference Type: Animal

Actual Object:
↓
Dog → Dog.sound()

Actual Object:
↓
Cat → Cat.sound()

The compiler checks that `Animal` has a `sound()` method.

The JVM determines at **runtime** whether the object is a `Dog` or a `Cat` and calls the appropriate overridden method.

---

6. How does polymorphism help in Spring applications?

Polymorphism is used extensively in Spring to create **flexible, loosely coupled applications**.

Example 1: Dependency Injection

```
interface PaymentService {
    void pay();
}
```

```
class CreditCardPayment implements PaymentService {

        public void pay() {
        System.out.println("Paid using Credit Card");
        }
}

class UpiPayment implements PaymentService {

        public void pay() {
        System.out.println("Paid using UPI");
        }
}
```

Service class:

```
class OrderService {

        private PaymentService paymentService;

        OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
        }

        void checkout() {
        paymentService.pay();
        }
}
```

The `OrderService` depends on the **interface**, not a specific implementation.

Spring can inject:

CreditCardPayment

or

UpiPayment

without changing the `OrderService` code.

Example 2: Bean Injection

```
interface NotificationService {
  void send();
}

class EmailService implements NotificationService {

        public void send() {
        System.out.println("Email Sent");
        }
}

class SmsService implements NotificationService {

        public void send() {
        System.out.println("SMS Sent");
        }
}
```

Spring can inject either implementation:

```
NotificationService service;
```

At runtime, the injected bean may be:

- `EmailService`
- `SmsService`

The client code remains unchanged because it works with the interface.

Benefits in Spring

- **Loose coupling**: Classes depend on abstractions (interfaces) instead of concrete implementations.
- **Easy to extend**: Add a new implementation without modifying existing client code (supports the Open/Closed Principle).
- **Dependency Injection**: Spring injects different implementations based on configuration or annotations.
- **Easier testing**: Replace real implementations with mock or stub implementations in unit tests.

- **Maintainability**: Business logic stays independent of implementation details.

Quick Revision

- **Polymorphism** allows the same method call to produce different behavior depending on the object's actual type.
- **Overloading**: Same method name, different parameter lists, resolved at compile time.
- **Overriding**: Child class provides a new implementation of a parent method, resolved at runtime.
- **Static methods cannot be overridden**; they are **hidden** because they belong to the class.
- **Private methods cannot be overridden** because they are not inherited.
- **Dynamic Method Dispatch** is the JVM's runtime mechanism for selecting the correct overridden method based on the actual object's type.
- In **Spring**, polymorphism enables dependency injection, loose coupling, interchangeable implementations, and easier testing.

---

1. What is Abstraction?

Abstraction is the process of **hiding implementation details and exposing only the essential functionality**.

The user knows **what** an object does, but not necessarily **how** it does it.

Java achieves abstraction using:

- **Abstract classes**
- **Interfaces**

Real-Life Example

A car driver uses:

- Steering wheel
- Brake
- Accelerator

The driver doesn't need to know how the engine or transmission works internally.

This is abstraction.

Example

```
abstract class Animal {

        abstract void sound();

        void sleep() {
        System.out.println("Sleeping...");
        }
}

class Dog extends Animal {

        @Override
        void sound() {
        System.out.println("Dog barks");
        }
}

public class Main {

        public static void main(String[] args) {

        Animal a = new Dog();

      a.sound();
        a.sleep();
        }
}
```

Output

Dog barks
Sleeping...

The user calls `sound()` without worrying about its implementation.

---

2. Difference between Abstract Class and Interface

Feature	Abstract Class	Interface
Keyword	`abstract class`	`interface`
Constructors	✅ Yes	❌ No
Instance Variables	✅ Yes	❌ No (only `public static final` constants)
Abstract Methods	✅ Yes	✅ Yes
Concrete Methods	✅ Yes	✅ Yes (`default`, `static`, and `private` methods)
Multiple Inheritance	❌ No	✅ Yes (multiple interfaces can be implemented)
State	Can maintain object state	Cannot maintain instance state
Relationship	Strong "is-a" relationship with shared implementation	Contract defining capabilities

---

Abstract Class Example

```
abstract class Vehicle {

    String brand;

    Vehicle(String brand) {
    this.brand = brand;
    }

    abstract void start();
```

```
    void stop() {
    System.out.println("Vehicle Stopped");
    }
}
```

Interface Example

```
interface Payment {

    void pay();

    default void printReceipt() {
    System.out.println("Receipt Printed");
    }
}
```

---

3. Can an Abstract Class Have Constructors?

Yes.

Although you cannot create an object of an abstract class directly, its constructor is called **when a subclass object is created**.

Example

```
abstract class Animal {

    Animal() {
    System.out.println("Animal Constructor");
    }
}

class Dog extends Animal {

    Dog() {
    System.out.println("Dog Constructor");
    }
}
```

```
public class Main {

	public static void main(String[] args) {
	Dog d = new Dog();
	}
}
```

Output

```
Animal Constructor
Dog Constructor
```

Why?

The abstract class constructor initializes the common state shared by all subclasses.

---

4. Can an Interface Have Default Methods?

Yes. Since **Java 8**, interfaces can contain **default methods**.

Default methods provide a method implementation that implementing classes may use or override.

Example

```
interface Vehicle {

	void start();

	default void stop() {
	System.out.println("Vehicle Stopped");
	}
}

class Car implements Vehicle {

	@Override
	public void start() {
	System.out.println("Car Started");
	}
}
```

```
public class Main {

        public static void main(String[] args) {

      Car car = new Car();

        car.start();
        car.stop();
        }
}
```

Output

```
Car Started
Vehicle Stopped
```

Interfaces can also contain:

```
interface Example {

        void method();            // Abstract

        default void test() { } // Default

        static void show() { }  // Static

        private void helper() { } // Private (Java 9+)
}
```

---

5. When Would You Use an Interface Over an Abstract Class?

Use an **interface** when you want to define a **contract** that multiple unrelated classes can implement.

Example

```
interface PaymentService {

        void pay();
}
```

Implementations:

```
class CreditCardPayment implements PaymentService {

        public void pay() {
        System.out.println("Credit Card Payment");
        }
}

class UpiPayment implements PaymentService {

        public void pay() {
        System.out.println("UPI Payment");
        }
}
```

Each payment type implements the same contract.

Use an Interface When

- Multiple unrelated classes should have the same capability.
- Multiple inheritance of type is needed.
- You want loose coupling.
- You're designing APIs or service contracts.

Use an Abstract Class When

- Classes share common fields and implementation.
- You want to provide base functionality.
- You need constructors or instance variables.

Example:

```
abstract class Employee {

        String name;
        double salary;
```

```
    void login() {
    System.out.println("Logged In");
    }

    abstract void work();
}
```

---

6. How is Abstraction Used in Spring Boot Services and Repositories?

Spring Boot uses abstraction extensively through **interfaces**.

Service Layer

Interface

```
public interface PaymentService {

    void pay();
}
```

Implementation

```
public class CreditCardPaymentService implements PaymentService {

    @Override
    public void pay() {
    System.out.println("Payment Successful");
    }
}
```

Usage

```
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
    }
```

```
        public void checkout() {
        paymentService.pay();
        }
}
```

`OrderService` depends on the **interface**, not the implementation.

Spring injects the appropriate implementation.

Repository Layer

You often define only an interface:

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

Notice there is **no implementation class**.

Spring Data JPA generates the implementation automatically at runtime.

Your code uses:

studentRepository.save(student);

studentRepository.findById(1L);

studentRepository.findAll();

without knowing how these methods are implemented.

This is abstraction in practice.

Benefits in Spring Boot

- **Loose coupling**: Business logic depends on interfaces, not concrete classes.
- **Easy to replace implementations**: Swap one implementation for another without changing client code.
- **Better testing**: Mock interfaces during unit tests.

- **Framework-generated implementations**: Spring Data JPA provides repository implementations automatically.
- **Cleaner architecture**: Controllers, services, and repositories communicate through well-defined contracts.

Quick Revision

- **Abstraction** hides implementation details and exposes only the required functionality.
- **Abstract class**: Can have constructors, fields, abstract methods, and concrete methods. Suitable for sharing common state and behavior.
- **Interface**: Defines a contract. Since Java 8, it can also contain `default` and `static` methods (and `private` methods since Java 9).
- **Abstract class constructors** are executed when a subclass object is created.
- **Default methods** allow interfaces to evolve without breaking existing implementations.
- Use an **interface** for contracts and loose coupling; use an **abstract class** when subclasses need to share common state or implementation.
- **Spring Boot** relies heavily on abstraction by programming to interfaces (services and repositories), enabling dependency injection, easier testing, and flexible implementations.

---

1. What is the use of `final`?

The `final` keyword is used to **restrict modification**.

It can be applied to:

- Variables

- Methods
- Classes

Depending on where it is used, it prevents:

- Reassigning a variable
- Overriding a method
- Extending a class

---

2. Difference between `final` Variable, `final` Method, and `final` Class

`final` Variable	`final` Method	`final` Class
Cannot be reassigned after initialization	Cannot be overridden by subclasses	Cannot be inherited (extended)
Used for constants or fixed references	Used to preserve method behavior	Used to prevent inheritance

---

A. Final Variable

Once assigned, it cannot be assigned again.

```
public class Main {

        public static void main(String[] args) {

        final int MAX = 100;

        // MAX = 200; // Compile-time error

        System.out.println(MAX);
        }
}
```

Output

B. Final Method

```
class Animal {

        final void eat() {
        System.out.println("Eating...");
        }
}

class Dog extends Animal {

        // void eat() { } // Compile-time error
}
```

A child class cannot override a `final` method.

C. Final Class

```
final class Car {

}

// class SportsCar extends Car { } // Compile-time error
```

No class can extend `Car`.

---

3. Can Final Objects Be Modified?

Yes.

A `final` reference variable cannot point to another object, but the object's internal state can still change (unless the object itself is immutable).

Example

```
class Student {
```

```
        String name;
}

public class Main {

        public static void main(String[] args) {

      final Student s = new Student();

        s.name = "Alice";       // Allowed

        System.out.println(s.name);

        // s = new Student();  // Compile-time error
        }
}
```

Output

Alice

Memory Illustration

Before:

```
s ————————► Student Object
        name = null
```

After:

```
s ————————► Student Object
        name = Alice
```

Changing:

```
s.name = "Bob";
```

is allowed because the object stays the same.

Changing:

```
s = new Student();
```

is **not allowed** because it changes the reference.

---

4. Why is `String` Final in Java?

The `String` class is declared as `final`.

```
public final class String {
        ...
}
```

Reasons

1. Security

Many sensitive values are stored as strings.

Examples:

- Passwords
- File paths
- URLs
- Database connection strings

If `String` were not `final`, someone could extend it and change its behavior.

---

2. Immutability

Strings are immutable.

```
String s = "Java";

s.concat(" Programming");

System.out.println(s);
```

Output

```
Java
```

The original string never changes.

---

3. Thread Safety

Immutable objects are naturally thread-safe because their state cannot change after creation.

---

4. String Pool Optimization

Java stores string literals in the **String Pool**.

String a = "Hello";
String b = "Hello";

Both variables can reference the same object safely because strings are immutable.

Summary

`String` is `final` to ensure:

- Security
- Immutability
- Thread safety
- Safe string pooling and optimizations

---

5. Can Constructors Be Final?

No.

Constructors **cannot** be declared `final`.

Example:

```
class Student {

    // final Student() { } // Compile-time error
}
```

Why?

A constructor is **not inherited**.

Only inherited methods can be overridden.

Since constructors cannot be overridden, making them `final` has no meaning.

Constructor vs Method

```
class A {

        A() {
        }

        void show() {
        }
}
```

- `show()` can be overridden.
- `A()` cannot be overridden.

Therefore:

- `show()` can be `final`.
- `A()` cannot be `final`.

Real-World Examples

Final Variable

```
public static final double PI = 3.14159;
```

Constant value.

Final Method

```
class BankAccount {
```

```
    final void calculateInterest() {
    System.out.println("Standard interest calculation");
    }
}
```

Ensures subclasses cannot change the interest calculation logic.

Final Class

```
final class Utility {

    public static int square(int x) {
    return x * x;
    }
}
```

Prevents inheritance when the class is not designed to be extended.

Quick Revision

- `final` restricts modification.
- **Final variable** → Cannot be reassigned after initialization.
- **Final method** → Cannot be overridden by subclasses.
- **Final class** → Cannot be extended.
- **Final object reference** → The reference cannot change, but the object's fields can still be modified unless the object is immutable.
- `String` **is** `final` to guarantee immutability, security, thread safety, and efficient string pooling.
- **Constructors cannot be** `final` because they are never inherited or overridden.

OOP Principles with a Banking Example

Object-Oriented Programming (OOP) is based on four main principles:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

4.

---

1. Encapsulation

Definition:
Encapsulation means **binding data and methods together** and **hiding the internal data** using access modifiers (`private`, `protected`, `public`).

Banking Example

```
class BankAccount {

    private double balance;

    public void deposit(double amount) {
    balance += amount;
    }

    public void withdraw(double amount) {
    if (balance >= amount) {
    balance -= amount;
    }
    }

    public double getBalance() {
    return balance;
    }
}
```

Usage:

```
BankAccount account = new BankAccount();

account.deposit(5000);
account.withdraw(1000);

System.out.println(account.getBalance());
```

Output

```
4000.0
```

Why Encapsulation?

- Prevents direct modification of `balance`.
- Validation can be added inside methods.
- Improves security and maintainability.

---

2. Abstraction

Definition:
Abstraction means **showing only essential features while hiding implementation details**.

Banking Example

```
abstract class Payment {

	abstract void pay(double amount);
}
class UPIPayment extends Payment {

	@Override
	void pay(double amount) {
	System.out.println("Paid ₹" + amount + " using UPI");
	}
}
```

Client code:

```
Payment payment = new UPIPayment();
payment.pay(1000);
```

The user only knows:

payment.pay(1000);

They don't need to know how authentication, transaction logging, or database updates happen.

---

3. Inheritance

Definition:
Inheritance allows one class to acquire properties and methods from another.

Banking Example

```
class Account {

	void deposit() {
	System.out.println("Deposited");
	}
}

class SavingsAccount extends Account {

	void calculateInterest() {
	System.out.println("Interest Calculated");
	}
}
```

Usage:

```
SavingsAccount account = new SavingsAccount();

account.deposit();
account.calculateInterest();
```

Output

```
Deposited
Interest Calculated
```

Here:

SavingsAccount is an Account

---

4. Polymorphism

Definition:
The same method call behaves differently depending on the actual object.

```
class Account {

	void calculateInterest() {
	System.out.println("General Interest");
	}
}
class SavingsAccount extends Account {

	@Override
	void calculateInterest() {
	System.out.println("Savings Interest");
	}
}
class CurrentAccount extends Account {

	@Override
	void calculateInterest() {
	System.out.println("No Interest");
	}
}
```

Usage:

```
Account account;

account = new SavingsAccount();
account.calculateInterest();

account = new CurrentAccount();
account.calculateInterest();
```

Output

```
Savings Interest
No Interest
```

Difference between Abstraction and Encapsulation

Abstraction	Encapsulation
Hides implementation details	Hides data
Focuses on **what** an object does	Focuses on **how** data is protected
Achieved using abstract classes and interfaces	Achieved using private fields and public methods
Simplifies usage	Improves security

Example

account.withdraw(1000);

- User doesn't know **how** withdrawal works → **Abstraction**
- User cannot directly modify `balance` → **Encapsulation**

Difference between Inheritance and Composition

Inheritance (Is-a)

```
class Account {
}

class SavingsAccount extends Account {
}
```

SavingsAccount is an Account

Uses:

- `extends`

Composition (Has-a)

```
class DebitCard {

	void swipe() {
	System.out.println("Card Swiped");
	}
}

class BankAccount {

	private DebitCard card = new DebitCard();

	void pay() {
	card.swipe();
	}
}
```

BankAccount has a DebitCard

Comparison

Inheritance	Composition
Is-a	Has-a
Uses `extends`	Uses object references
Tight coupling	Loose coupling
Less flexible	More flexible

Modern Java design generally **prefers composition over inheritance** unless there is a true "is-a" relationship.

When Would You Avoid Inheritance?

Avoid inheritance when:

1. There is no genuine is-a relationship

❌ Wrong

```
class Printer {
}

class Employee extends Printer {
}
```

An employee is **not** a printer.

---

2. Only code reuse is needed

Use composition instead.

---

3. You want loose coupling

Inheritance tightly couples child classes to parent classes.

---

4. Parent behavior may change independently

Example:

Car has an Engine

Not

Car is an Engine

Composition is a better fit.

What is Runtime Polymorphism?

Runtime polymorphism occurs when the JVM decides **at runtime** which overridden method to execute.

```
class Account {
```

```
        void display() {
        System.out.println("Account");
        }
}

class SavingsAccount extends Account {

        @Override
        void display() {
        System.out.println("Savings");
        }
}
Account account = new SavingsAccount();

account.display();
```

Output

Savings

The compiler checks that `display()` exists in `Account`, while the JVM calls the overridden method based on the actual object (`SavingsAccount`).

How Does Method Overriding Work Internally?

When you write:

Account account = new SavingsAccount();

account.display();

The process is:

1. **Compilation**
    - The compiler verifies that `Account` has a `display()` method.
2. **Runtime**
    - The JVM looks at the **actual object** (`SavingsAccount`).

- It invokes `SavingsAccount.display()`.

This runtime selection is called **Dynamic Method Dispatch**, which enables runtime polymorphism.

Can Static Methods Be Overridden?

No.

Static methods belong to the **class**, not to objects.

If a child declares a static method with the same signature, it **hides** the parent's method.

```
class Parent {

	static void show() {
	System.out.println("Parent");
	}
}

class Child extends Parent {

	static void show() {
	System.out.println("Child");
	}
}

public class Main {

	public static void main(String[] args) {
	Parent p = new Child();
	p.show();
	}
}
```

Output

Parent

This is **method hiding**, not overriding.

Can Private Methods Be Overridden?

No.

Private methods are **not inherited**, so subclasses cannot override them.

```
class Parent {

	private void show() {
	System.out.println("Parent");
	}
}

class Child extends Parent {

	void show() {
	System.out.println("Child");
	}
}
```

Here, `Child.show()` is a **new method**, not an overridden one.

Difference between Abstract Class and Interface

Feature	Abstract Class	Interface
Constructors	✅ Yes	❌ No
Instance Variables	✅ Yes	❌ No (only constants)
Abstract Methods	✅ Yes	✅ Yes
Concrete Methods	✅ Yes	✅ Yes (`default`, `static`, `private`)
Multiple Inheritance	❌ No	✅ Yes
Purpose	Share common state and behavior	Define a contract

Abstract Class

```
abstract class Account {

	String accountNumber;

	abstract void withdraw();

	void deposit() {
	System.out.println("Deposited");
	}
}
```

Interface

```
interface PaymentService {

	void pay();

	default void printReceipt() {
	  System.out.println("Receipt Printed");
	}
}
```

Can an Abstract Class Have a Constructor?

Yes.

Although you cannot instantiate an abstract class directly, its constructor is executed when a subclass object is created.

```
abstract class Account {

	Account() {
	System.out.println("Account Constructor");
	}
}

class SavingsAccount extends Account {

	SavingsAccount() {
```

```
        System.out.println("Savings Constructor");
        }
}

public class Main {

        public static void main(String[] args) {
        new SavingsAccount();
        }
}
```

Output

Account Constructor
Savings Constructor

The abstract class constructor initializes common state before the subclass constructor runs.

Quick Revision

- **Encapsulation**: Protect data using private fields and controlled access through methods.
- **Abstraction**: Hide implementation details and expose only essential operations.
- **Inheritance**: Reuse and specialize behavior using an **is-a** relationship.
- **Composition**: Build classes using a **has-a** relationship; generally preferred for flexibility.
- **Polymorphism**: The same method call behaves differently depending on the actual object.
- **Runtime polymorphism** is achieved through **method overriding** and **dynamic method dispatch**.
- **Static methods cannot be overridden**; they are hidden.
- **Private methods cannot be overridden** because they are not inherited.
- **Abstract classes** can have constructors, fields, and implemented methods; **interfaces** define contracts and support multiple inheritance of type.

---

1. Why is `String` immutable?

A `String` object **cannot be modified after it is created**.

String s = "Java";
s.concat(" Programming");

System.out.println(s);

Output:

Java

A new `String` object is created by `concat()`, but `s` still refers to the original object.

Why Java made `String` immutable?

1. Security

Used for

- Database URLs
- File paths
- Class loading
- Network connections

If mutable:

"jdbc:mysql://prod-db"

could be changed by another object.

---

2. Thread Safety

Immutable objects can be safely shared.

String message = "Hello";

100 threads can read it simultaneously.

---

3. String Pool

Immutable objects allow safe sharing.

String a = "Java";
String b = "Java";

Both point to same object.

---

4. HashMap Keys

Hash code never changes.

Map<String,Integer>

works efficiently.

---

2. Explain the String Constant Pool

The JVM maintains a special memory area called the **String Constant Pool (SCP)**.

String s1 = "Java";
String s2 = "Java";

Memory

```
SCP

+---------+
| "Java"  |
+---------+
↑         ↑
s1        s2
```

Only one object is created.

Using `new`

String s1 = new String("Java");
String s2 = new String("Java");

Memory

Pool

"Java"

Heap

Object1
Object2

Two heap objects.

Benefits

- Saves memory
- Faster comparisons
- Better performance

---

3. Difference between String, StringBuilder and StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable	❌	✅	✅
Thread Safe	Yes (immutable)	No	Yes
Speed	Slow for modifications	Fastest	Slower than Builder

Synchronization	Not needed	No	Yes

String

```
String s = "Hello";
s += " World";
```

Creates new object.

StringBuilder

```
StringBuilder sb = new StringBuilder();

sb.append("Hello");
sb.append(" World");
```

Same object modified.

StringBuffer

Same as Builder but synchronized.

Used in multithreaded code.

---

4. What is `intern()`?

`intern()` returns the pooled version of a String.

```
String s1 = new String("Java");
String s2 = s1.intern();

System.out.println(s2 == "Java");
```

Output

true

Use when

- Millions of repeated Strings
- Memory optimization
- Caching repeated values

---

5. Difference between `==` and `equals()`

==

Compares references.

equals()

Compares content.

Example

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a==b);
System.out.println(a.equals(b));
```

Output

```
false
true
```

---

6. Contract between `equals()` and `hashCode()`

Rule 1

If

a.equals(b)

is true

then

a.hashCode()==b.hashCode()

must also be true.

Rule 2

Unequal objects may have same hashCode.

---

7. How does HashMap use hashCode()?

Steps

put(key,value)

↓

hashCode()

↓

bucket index

↓

equals()

↓

store/retrieve value

Example

map.put(student,value);

HashMap

student.hashCode()

↓

bucket

↓

equals()

↓

find exact key

---

8. Why immutable objects should be HashMap keys?

Suppose

Employee e=new Employee(1);

used as key.

Later

e.id=2;

Hash code changes.

HashMap searches wrong bucket.

Object becomes unreachable.

Immutable keys prevent this.

Examples

- String
- Integer
- UUID

---

9. How to create custom immutable class?

Rules

- class final
- fields private final
- no setters
- defensive copy for mutable objects

Example

```
public final class Employee{

        private final int id;
        private final String name;

        public Employee(int id,String name){
        this.id=id;
        this.name=name;
        }

        public int getId(){
         return id;
        }

        public String getName(){
        return name;
        }
}
```

---

10. Wrapper classes and Autoboxing

Wrapper classes

int → Integer
double → Double
boolean → Boolean

Autoboxing

Integer x=10;

Compiler

Integer.valueOf(10)

Unboxing

Integer x=20;

int y=x;

Compiler

x.intValue()

---

11. Can enum have constructors, methods and implement interfaces?

Yes.

Example

```
interface Printable{
        void print();
}

enum Status implements Printable{

        SUCCESS,
        FAILED;

        Status(){
        System.out.println("Created");
        }

        public void print(){
      System.out.println(this);
        }
}
```

Cannot extend classes because enum already extends `java.lang.Enum`.

---

12. Explain `@Override`, `@FunctionalInterface`, and `@Retention`

@Override

Checks that a method correctly overrides a superclass or interface method.

```
@Override
public void run(){}
```

@FunctionalInterface

Marks an interface with exactly one abstract method.

```
@FunctionalInterface
interface Calculator{
	int add(int a,int b);
}
```

Used with lambda expressions.

@Retention

Specifies how long an annotation is retained.

```
@Retention(RetentionPolicy.RUNTIME)
```

Common values:

- `SOURCE`
- `CLASS`
- `RUNTIME`

Spring annotations such as `@Component` and `@Autowired` use runtime retention so they can be discovered via reflection.

---

13. Types of Inner Classes

1. Member Inner Class
2. Static Nested Class
3. Local Inner Class
4. Anonymous Inner Class

Example

```
class Outer{
    class Inner{}
    static class Nested{}
}
```

---

14. Upcasting and Downcasting

Upcasting

Animal a=new Dog();

Automatic.

Downcasting

Dog d=(Dog)a;

Explicit.

Practical example

List list=new ArrayList();

The reference uses the interface (`List`), while the object is an `ArrayList`, making it easy to switch implementations later.

---

15. When can ClassCastException occur?

Example

Animal a=new Cat();

Dog d=(Dog)a;

Runtime

ClassCastException

Prevent

```
if(a instanceof Dog){
        Dog d=(Dog)a;
}
```

---

16. Why override `toString()`?

Default

Student@5f184fc6

Override

Student{id=1,name='John'}

Useful for

- Logging
- Debugging
- REST responses
- Monitoring

---

17. Common mistakes when overriding `equals()` and `hashCode()`

Mistakes

- Override `equals()` but not `hashCode()`
- Use mutable fields in `hashCode()`
- Use `==` instead of `equals()` for object fields
- Forget null checks
- Break symmetry or transitivity

Always override both methods together.

---

18. How does JVM optimize String memory?

- String Constant Pool
- String deduplication (supported by collectors such as G1 in modern JVMs)
- Reuse of string literals
- Compile-time constant folding

Example

```
String s1="Java";
String s2="Java";
```

Only one pooled object.

---

19. Performance issues with excessive String concatenation

Bad

```
String s="";

for(int i=0;i<10000;i++){

        s=s+i;
}
```

Creates thousands of temporary objects.

Better

```
StringBuilder sb=new StringBuilder();

for(int i=0;i<10000;i++){

        sb.append(i);
}
```

Much faster and more memory-efficient.

---

20. Enterprise / Spring Boot Examples

Immutable DTO

```
public record UserResponse(
        Long id,
        String name
) {}
```

Immutable objects are ideal for REST responses because they are thread-safe and cannot be accidentally modified.

HashMap Cache

```
Map<String, User> cache = new HashMap<>();
```

`String` is an excellent key because it is immutable and has a stable `hashCode()`.

Enum for Order Status

```
public enum OrderStatus {
        PENDING,
        PAID,
        SHIPPED,
        DELIVERED
}
```

Using enums prevents invalid status values and makes the code more readable.

Interface and Polymorphism

```
public interface PaymentService {
        void pay();
}

@Service
class UpiPaymentService implements PaymentService {

        public void pay() {
           System.out.println("Paid via UPI");
        }
}
```

A controller or service depends on the `PaymentService` interface, and Spring injects the appropriate implementation. This demonstrates **abstraction**, **polymorphism**, and **dependency injection** working together.

Quick Revision

- **String** is immutable for security, thread safety, string pooling, and reliable hash codes.
- The **String Constant Pool** stores shared string literals to reduce memory usage.
- **StringBuilder** is mutable and fast; **StringBuffer** is mutable and thread-safe.
- `intern()` returns the pooled version of a string.
- `==` compares references; `equals()` compares logical content.

- If two objects are equal according to **`equals()`**, they **must** return the same **`hashCode()`**.
- **HashMap** uses `hashCode()` to find a bucket and `equals()` to locate the exact key.
- **Immutable objects** make safe and reliable `HashMap` keys.
- A custom immutable class should be `final`, use `private final` fields, expose no setters, and defensively copy mutable state.
- **Wrapper classes** allow primitives to be treated as objects; **autoboxing/unboxing** performs automatic conversions.
- **Enums** can have constructors, methods, fields, and implement interfaces.
- Common annotations include **`@Override`**, **`@FunctionalInterface`**, and **`@Retention`**.
- Inner classes include **member**, **static nested**, **local**, and **anonymous** classes.
- **Upcasting** is automatic; **downcasting** requires an explicit cast and should be guarded with `instanceof` when appropriate.
- Override **`toString()`** to improve logging and debugging.
- Excessive string concatenation in loops should use **`StringBuilder`** for better performance.
- These concepts are widely used in **Spring Boot** for DTOs, caching, enums, dependency injection, and enterprise application design.

---

1. String Internals

Q1. Why is String immutable in Java?

Answer

A String object cannot be changed after creation.

String s = "Hello";

s.concat(" World");

System.out.println(s);

Output

Hello

A new String object is created.

Why Java made String immutable?

1. Security

Database URLs, usernames, passwords cannot be modified.

String url = "jdbc:mysql://localhost";

If mutable, another thread could change it.

---

2. String Pool

Since String never changes, multiple variables can share one object.

String a = "Java";
String b = "Java";

Only one object exists.

---

3. Thread Safety

No synchronization required.

Multiple threads can safely use the same String.

---

4. HashMap Performance

String caches its hashCode.

HashMap<String,Integer>

HashMap doesn't recalculate hash every time.

Interview Tip

If interviewer asks

Can String be mutable?

Answer

No.

The String class is final and internally stores characters in a private final array, preventing modification after creation.

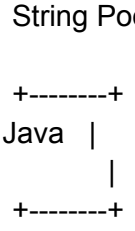
Q2. What is String Pool?

Answer

String Pool is a special memory area inside Heap that stores unique String literals.

```
String a="Java";
String b="Java";
```

Both point to same object.



Why String Pool?

To save memory.

Without pool

1000 "Java" Strings

↓

1000 objects

With pool

1000 references

↓

1 object

Q3. Difference between new String() and String literal?

String s1="Java";
String s2="Java";

String s3=new String("Java");

Memory

Pool

Java

Heap

new String("Java")

Objects

s1,s2 -> Pool object

s3 -> Heap object

Comparison

System.out.println(s1==s2);

true

System.out.println(s1==s3);

false

System.out.println(s1.equals(s3));

true

Q4. What is intern()?

String s=new String("Java");

String t=s.intern();

intern() returns pooled String.

Heap Object

↓

Pool Object

Q5. Why == should not be used for String comparison?

Because == compares references.

String a=new String("Java");
String b=new String("Java");
a==b

false
a.equals(b)

true

Q6. How is hashCode cached in String?

String calculates hashCode only once.

private int hash;

First call

hashCode()

↓

Calculate

↓

Store

Next call

Return cached value

Very common interview question.

Q7. What happens when you concatenate Strings?

String s="A";

s=s+"B";

Internally

new StringBuilder()

↓

append("A")

↓

append("B")

↓

toString()

Compiler converts it into StringBuilder.

---

2. StringBuilder

Q8. Why StringBuilder?

Because String is immutable.

Bad

```
for(...)
{
s=s+i;
}
```

Creates thousands of objects.

Good

```
StringBuilder sb=new StringBuilder();

sb.append(i);
```

Q9. Difference between StringBuilder and StringBuffer?

StringBuilder	StringBuffer

Not synchronized	Synchronized
Faster	Slower
Java 5	Java 1.0
Single Thread	Multi Thread

Q10. Which one should you use?

Single thread

StringBuilder

Multiple threads

StringBuffer

Q11. Does StringBuilder override equals()?

No.

StringBuilder a=new StringBuilder("Java");

StringBuilder b=new StringBuilder("Java");
a.equals(b)

false

Because Object's equals() compares reference.

Object Class

Q12. Why every class extends Object?

Because Object is root class.

Employee

↓

Person

↓

Object

Every class gets methods like

equals()

hashCode()

toString()

wait()

notify()

clone()

Q13. Most important Object class methods?

equals()

hashCode()

toString()

clone()

wait()

notify()

notifyAll()

getClass()

---

equals()

Q14. Difference between == and equals()?

==

Reference comparison

equals()

Content comparison

Example

String a="Java";

String b=new String("Java");
==

false

equals

true

Q15. Why override equals()?

Suppose

Employee

id

name

Two employees

id=10

name=John

Should be equal.

Default equals()

Reference comparison

false

Override

Content comparison

true

---

hashCode()

Q16. Why override hashCode when equals is overridden?

HashMap first checks

hashCode()

↓

Bucket

↓

equals()

If hashCode isn't overridden

Equal objects go into different buckets.

Interview favorite.

Q17. hashCode contract?

If

equals()

true

Then

hashCode()

must be same

Reverse isn't mandatory.

---

toString()

Q18. Why override toString()?

Default

Employee@45ff54

Override

Employee{id=1,name=John}

Useful for logging.

Wrapper Classes

Q19. Why wrapper classes?

Collections store objects.

List<int>

❌

Need

List<Integer>

✔

Q20. What is Autoboxing?

Primitive

↓

Wrapper

Integer x=10;

Compiler converts

Integer.valueOf(10)

Q21. What is Unboxing?

Integer x=100;

int y=x;

Compiler

x.intValue()

Immutable Objects

Q22. How to make immutable class?

Rules

final class

private fields

no setters

constructor initialization

defensive copy

Q23. Why immutable objects are thread safe?

Because state never changes.

No synchronization required.

Enums

Q24. Why Enum instead of constants?

Old

public static final int RED=1;

Problems

No type safety.

Enum

enum Color

{

RED,GREEN,BLUE

}

Type safe.

Q25. Can Enum have constructor?

Yes.

enum Status{

ACTIVE(1),

INACTIVE(2);

private int value;

Status(int value){

this.value=value;

}

}

Q26. Can Enum implement interface?

Yes.

Can extend class?

No.

Already extends

java.lang.Enum

Annotations

Q27. What are annotations?

Metadata for compiler/runtime.

Examples

@Override

@Deprecated

@SuppressWarnings

@Entity

@Service

Q28. Difference between @Retention policies?

SOURCE

Removed after compilation.

CLASS

Stored in class file.

RUNTIME

Available via Reflection.

Interview favorite.

Inner Classes

Q29. Types of Inner Classes?

1.

Member Inner

2.

Static Nested

3.

Local Inner

4.

Anonymous Inner

Q30. Difference between Inner class and Static Nested class?

Inner class

Requires outer object.

Outer.Inner

Static Nested

No outer object required.

Outer.StaticNested

Q31. When Anonymous class is preferred?

Before Java 8

Runnable r=new Runnable(){

public void run(){}

};

After Java 8

Prefer Lambda.

Upcasting

Q32. What is Upcasting?

Animal a=new Dog();

Child

↓

Parent

Automatic.

Advantages

Loose coupling

Runtime Polymorphism

Downcasting

Q33. What is Downcasting?

Animal a=new Dog();

Dog d=(Dog)a;

Needed to access child methods.

Q34. Why instanceof before downcasting?

Avoid

ClassCastException
if(a instanceof Dog){

Dog d=(Dog)a;

}

Q35. Can every parent object be downcast?

No.

Animal a=new Animal();

Dog d=(Dog)a;

Runtime

ClassCastException

Rapid-Fire Questions (Very Frequently Asked)

1. Why is `String` immutable?
2. Where is the String Pool stored?
3. Difference between `StringBuilder` and `StringBuffer`.
4. Why should `equals()` and `hashCode()` be overridden together?
5. Difference between `==` and `equals()`.
6. What is the hashCode contract?
7. Why override `toString()`?
8. Explain autoboxing and unboxing.
9. How do you create an immutable class?
10. Why are immutable objects thread-safe?

11. Can an `enum` have constructors and methods?
12. Can an `enum` extend another class?
13. What are the retention policies of annotations?
14. What are the different types of inner classes?
15. What is the difference between an inner class and a static nested class?
16. Explain upcasting and downcasting with examples.
17. Why use `instanceof` before downcasting?
18. What does `String.intern()` do?
19. Why is `StringBuilder` faster than `StringBuffer`?
20. Why is `StringBuilder.equals()` different from `String.equals()`?

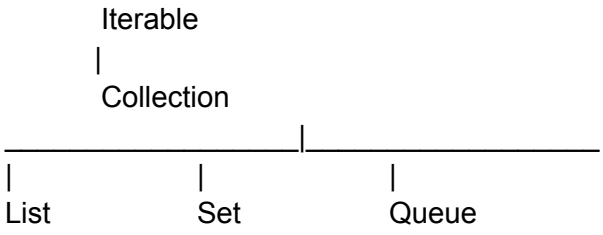
Java Collections Framework

Q1. What is the Java Collections Framework?

Answer

The **Java Collections Framework (JCF)** is a set of interfaces and classes that provide reusable data structures and algorithms to store and manipulate groups of objects.

Main Interfaces



Map
(Separate Hierarchy)

Benefits

- Dynamic size
- Ready-made data structures
- Sorting
- Searching
- Thread-safe variants
- Generic support

LIST

Q2. What is List?

Answer

A **List** is an ordered collection.

Characteristics

- Maintains insertion order
- Allows duplicates
- Index based
- Multiple null values allowed

Example

```
List<String> list = new ArrayList<>();

list.add("Java");
list.add("Python");
list.add("Java");
```

System.out.println(list);

Output

[Java, Python, Java]

Q3. Difference between ArrayList and LinkedList?

Feature	ArrayList	LinkedList
Internal Structure	Dynamic Array	Doubly Linked List
Random Access	Fast O(1)	Slow O(n)
Insert Middle	Slow	Fast
Delete Middle	Slow	Fast
Memory	Less	More
Cache Friendly	Yes	No

Interview Answer

Use **ArrayList** when reading data frequently.

Use **LinkedList** when inserting/deleting frequently.

Q4. When does ArrayList resize?

Default capacity

10

When full

newCapacity = old + old/2

Example

10

↓

15

↓

22

↓

33

Q5. Why is ArrayList random access fast?

Because it stores elements in a contiguous array.

Index

0 1 2 3 4

↓

Array

Address calculation

Base + (index × elementSize)

Hence

O(1)

LinkedList

Q6. How does LinkedList work?

Each node contains

Previous

Data

Next

Example

null

↓

10 ⇄ 20 ⇄ 30

↓

null

Insertion only changes pointers.

Q7. Why is LinkedList slower for searching?

Because traversal starts from head or tail.

Need to visit nodes one by one.

10

↓

20

↓

30

↓

40

Time

O(n)

SET

Q8. What is Set?

Characteristics

- No duplicates
- Unordered (HashSet)
- One null allowed (HashSet)

Example

```
Set<Integer> set = new HashSet<>();

set.add(10);
set.add(10);
set.add(20);

System.out.println(set);
```

Output

[10,20]

Q9. Difference between HashSet and TreeSet?

HashSet	TreeSet
Hash Table	Red Black Tree
Unordered	Sorted
O(1)	O(log n)
One null	No null
Faster	Slower

Q10. When should TreeSet be used?

When automatic sorting is required.

Example

Names

↓

Alphabetical Order

QUEUE

Q11. Difference between Queue and Stack?

Queue

FIFO

First In First Out

Example

Ticket Counter

Stack

LIFO

Last In First Out

Example

Undo Operation

Q12. Difference between add() and offer()?

add()

Throws Exception

offer()

Returns false

Preferred in production.

MAP

Q13. What is Map?

Stores

Key

↓

Value

Rules

- Keys unique
- Values duplicate allowed
- One null key (HashMap)
- Multiple null values

Q14. Difference between HashMap and TreeMap?

HashMap	TreeMap
Hash Table	Red Black Tree
Unordered	Sorted

O(1)	O(log n)
One null key	No null key
Faster	Slower

Q15. Difference between HashMap and Hashtable?

HashMap	Hashtable
Not synchronized	Synchronized
Faster	Slower
Allows null	Doesn't allow null

Hashtable is considered legacy.

HASHMAP INTERNALS

Q16. How does HashMap work internally?

Interview Favorite

Steps

put(key,value)

↓

hashCode()

↓

Hash

↓

Bucket Index

↓

Store Entry

When retrieving

get(key)

↓

hashCode()

↓

Bucket

↓

equals()

↓

Return Value

Q17. Why are both hashCode() and equals() needed?

Suppose

Bucket 3

↓

Employee

↓

Employee

↓

Employee

hashCode()

Finds bucket.

equals()

Finds exact object.

Without equals()

Cannot distinguish collisions.

Q18. What is Collision?

Different keys

Same bucket

Hash

↓

Bucket 5

↓

A

↓

B

↓

C

Q19. What happens after Java 8 when many collisions occur?

Before Java 8

Linked List

After Java 8

If bucket size >= 8

↓

Red Black Tree

Called

Treeification

Q20. What is Treeify Threshold?

8

When bucket size becomes 8

Linked List

↓

Red Black Tree

Q21. What is Untreeify Threshold?

6

When elements reduce below 6

Tree

↓

Linked List

Q22. Why treeify at 8 instead of 2 or 3?

Because maintaining a Red-Black Tree has overhead. A linked list is faster for a small number of elements. After around **8** nodes, the tree's **O(log n)** lookup generally outweighs the tree maintenance cost.

Q23. Why minimum capacity should be 64 before treeification?

If table size

<64

HashMap first

Resize

Instead of treeify.

Reason

Better distribution usually removes collisions.

Comparable

Q24. Difference between Comparable and Comparator?

Comparable	Comparator
Natural Sorting	Custom Sorting
compareTo()	compare()
Inside Class	Outside Class
One Sorting	Multiple Sorting

Q25. When should Comparable be used?

If every Employee should always be sorted by

Employee Id

Implement Comparable.

```
class Employee implements Comparable<Employee> {
        @Override
        public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);
        }
}
```

Q26. When should Comparator be used?

Need multiple sorting

Salary

↓

Age

↓

Name

Each Comparator is separate.

Q27. compareTo() contract?

Negative

↓

Current < Other

Zero

↓

Equal

Positive

↓

Current > Other

GENERICS

Q28. Why Generics?

Before Java 5

List list = new ArrayList();

list.add("Java");

Integer x = (Integer) list.get(0);

Runtime Exception

ClassCastException

Generics solve this.

List<String> list = new ArrayList<>();

Compile-time safety.

Q29. What is Type Erasure?

Generics exist only at compile time.

Compiler converts

List<String>

to

List

at runtime.

Q30. Why can't we create Generic arrays?

new T[10]

Compilation Error.

Reason

Type information is erased.

Q31. Difference between <?>, <? extends T>, and <? super T>?

Wildcard	Meaning	Can Read	Can Write
`<?>`	Unknown type	Yes (`Object`)	No
`<? extends Number>`	Upper bounded	Yes	No (except `null`)
`<? super Integer>`	Lower bounded	As `Object`	Yes (`Integer` and subclasses)

Remember the **PECS** rule:

- **Producer Extends**
- **Consumer Super**

Q32. What is fail-fast vs fail-safe?

Fail-fast

- `ArrayList`
- `HashMap`

- `HashSet`

Throws

ConcurrentModificationException

if modified during iteration.

Fail-safe

- `CopyOnWriteArrayList`
- `ConcurrentHashMap`

Works on a copy or concurrent structure.

Q33. What is the difference between Iterator and ListIterator?

Iterator	ListIterator
Forward only	Forward & backward
List, Set, Queue	List only
Remove	Add, Remove, Set

Q34. Why is HashMap not thread-safe?

Multiple threads can simultaneously modify its internal buckets, leading to race conditions and inconsistent state. Use `ConcurrentHashMap` or synchronize externally if concurrent updates are required.

JDBC Architecture

Q1. What is JDBC?

Answer

JDBC (Java Database Connectivity) is a standard Java API used to connect Java applications with relational databases such as MySQL, Oracle, PostgreSQL, SQL Server, etc.

It allows Java applications to:

- Connect to a database
- Execute SQL queries
- Retrieve results
- Perform CRUD operations
- Manage transactions

Q2. Explain JDBC Architecture.

Interview Favorite

Java Application
|
|
JDBC API
(Connection, Statement, ResultSet)
|
|
DriverManager
|
|
JDBC Driver
(MySQL Driver / Oracle Driver)
|
|
Database

Flow

1. Java application requests a connection.
2. DriverManager loads the JDBC driver.
3. Driver establishes a connection with the database.
4. SQL query is executed.
5. Database returns results.
6. JDBC converts results into Java objects.

Q3. What are the steps to connect Java with a database?

1. Load Driver (Optional in JDBC 4+)
2. Get Connection
3. Create Statement
4. Execute Query
5. Process Result
6. Close Resources

Example

```
Connection con = DriverManager.getConnection(url, user, password);

PreparedStatement ps =
        con.prepareStatement("SELECT * FROM employee");

ResultSet rs = ps.executeQuery();

while(rs.next()) {
        System.out.println(rs.getString("name"));
}
```

DriverManager

Q4. What is DriverManager?

Answer

`DriverManager` is a factory class responsible for managing JDBC drivers and creating database connections.

Example

Connection con =
DriverManager.getConnection(url,user,password);

Responsibilities

- Finds suitable JDBC driver
- Creates database connection
- Manages registered drivers

Q5. Is Class.forName() still required?

Answer

Before JDBC 4

Class.forName("com.mysql.cj.jdbc.Driver");

Required.

After JDBC 4

Not required.

The driver is automatically loaded using the **Service Provider Mechanism (SPI)** if the driver JAR is present.

Interview Tip

If asked:

Why do many projects still use Class.forName()?

Answer:

- Backward compatibility
- Explicit driver loading
- Older codebases

Connection

Q6. What is Connection?

Represents an active session between Java and the database.

Responsibilities

- Create Statements
- Commit transactions
- Rollback transactions
- Close connection

Example

```
Connection con =
DriverManager.getConnection(url,user,password);
```

Q7. Why should Connection be closed?

Database connections are expensive.

If not closed:

- Connection leaks
- Pool exhaustion
- Memory issues
- Application slowdown

Always use

try-with-resources

Statement

Q8. What is Statement?

Executes static SQL.

Example

```
Statement st = con.createStatement();

ResultSet rs =
st.executeQuery("SELECT * FROM employee");
```

Q9. Disadvantages of Statement?

Major interview question.

Problems

- SQL Injection
- No pre-compilation
- Slow for repeated execution

PreparedStatement

Q10. Why is PreparedStatement preferred?

Interview Favorite

Example

```
PreparedStatement ps =
con.prepareStatement(
"SELECT * FROM employee WHERE id=?");

ps.setInt(1,101);
```

Advantages

- Prevents SQL Injection
- Faster
- Precompiled SQL
- Easy parameter binding
- Better readability

Q11. What is SQL Injection?

Suppose user enters

```
admin' OR '1'='1
```

Using Statement

```
String sql =
"SELECT * FROM users WHERE username='"
+ user + "'";
```

Generated SQL

```
SELECT *
FROM users
WHERE username='admin'
OR '1'='1';
```

Returns all users.

PreparedStatement prevents this.

Q12. Why is PreparedStatement faster?

Because SQL is parsed and compiled once.

Execution flow

Prepare SQL

↓

Execution Plan

↓

Reuse

↓

Execute Multiple Times

Statement compiles every execution.

CallableStatement

Q13. What is CallableStatement?

Used to execute

- Stored Procedures
- Stored Functions

Example

```
CallableStatement cs =
con.prepareCall("{call getEmployee(?)}");

cs.setInt(1,100);

ResultSet rs = cs.executeQuery();
```

Q14. Difference between Statement, PreparedStatement and CallableStatement?

Statement	PreparedStatement	CallableStatement
Static SQL	Parameterized SQL	Stored Procedure
Slow	Faster	Procedure execution
SQL Injection possible	Safe	Safe
No pre-compilation	Precompiled	Procedure already compiled

ResultSet

Q15. What is ResultSet?

Represents data returned by SELECT.

Example

```
while(rs.next()){

System.out.println(rs.getString("name"));

}
```

Cursor initially points

Before First Row

Q16. Difference between execute(), executeUpdate(), executeQuery()?

Method	Used For	Returns

executeQuery()	SELECT	ResultSet
executeUpdate()	INSERT, UPDATE, DELETE	int
execute()	Unknown SQL / Mixed results	boolean

Transactions

Q17. What is Transaction?

Group of SQL operations executed as one unit.

Example

Transfer Money

Debit Account

↓

Credit Account

Both must succeed.

Q18. How to manage transactions?

```
con.setAutoCommit(false);

try{

// SQL

con.commit();

}catch(Exception e){

con.rollback();
```

}

Q19. What is AutoCommit?

Default

true

Each SQL commits automatically.

For multiple related queries

setAutoCommit(false)

Batch Processing

Q20. What is Batch Processing?

Instead of

Insert

Insert

Insert

Use

addBatch()

↓

executeBatch()

Reduces database round trips.

DAO Pattern

Q21. What is DAO?

DAO

Data Access Object

Purpose

Separate database logic from business logic.

Architecture

Controller

↓

Service

↓

DAO

↓

Database

Q22. Why DAO Pattern?

Advantages

- Clean architecture
- Easy testing
- Reusable
- Loose coupling
- Easier maintenance

Q23. What should DAO contain?

DAO contains only

- SQL
- JDBC
- CRUD methods

Should **NOT** contain

- Business logic
- Validation
- Calculations

Service Layer

Q24. What is Service Layer?

Contains business logic.

Example

Transfer Money

Validate Sender

↓

Validate Receiver

↓

Debit

↓

Credit

↓

Send Notification

DAO only executes SQL.

Q25. Difference between DAO and Service?

DAO	Service
Database logic	Business logic
SQL	Rules
CRUD	Validation
JDBC	Calls DAO

Factory Pattern

Q26. What is Factory Pattern?

Creates objects without exposing creation logic.

Instead of

```
EmployeeDAO dao =
new EmployeeDAOImpl();
```

Use

```
EmployeeDAO dao =
DAOFactory.getEmployeeDAO();
```

Benefits

- Loose coupling
- Easier testing

- Easy replacement of implementations

Q27. Where is Factory Pattern used in Java?

Common examples

- `DriverManager`
- `Calendar.getInstance()`
- `Executors.newFixedThreadPool()`
- `LoggerFactory.getLogger()`

Q28. Why is DriverManager considered a Factory?

Because it returns an appropriate `Connection` implementation without exposing which driver-specific class (MySQL, Oracle, PostgreSQL, etc.) is actually instantiated.

Q29. Why use try-with-resources?

Automatically closes

- Connection
- Statement
- PreparedStatement
- ResultSet

Example

```
try(Connection con = DriverManager.getConnection(url,user,password);
        PreparedStatement ps =
      con.prepareStatement("SELECT * FROM employee");
        ResultSet rs = ps.executeQuery()) {

        while(rs.next()) {
        System.out.println(rs.getString("name"));
        }
}
```

No finally block needed.

Performance Questions

Q30. How can JDBC performance be improved?

Expected interview answer:

- Use `PreparedStatement`
- Use Connection Pooling (HikariCP, Apache DBCP, etc.)
- Use Batch Processing
- Fetch only required columns
- Use indexes appropriately
- Close resources promptly
- Avoid unnecessary database calls
- Tune fetch size for large result sets

Q31. Why is PreparedStatement safer than Statement?

Because user input is sent separately from the SQL statement. The database treats input as data, not executable SQL, preventing SQL injection attacks.

Q32. Why should Connection objects not be created repeatedly?

Creating a database connection is expensive because it involves:

- Network communication
- Authentication
- Session creation
- Resource allocation

Use **connection pooling** instead.

Q33. What happens if commit() is not called when AutoCommit is false?

The transaction remains uncommitted. If the connection is closed without a commit, most databases roll back the pending transaction.

Q34. Can one Connection create multiple PreparedStatements?

Yes. A single `Connection` can create many `PreparedStatement` objects for different SQL statements.

Q35. Why should business logic not be written inside DAO?

The DAO should only interact with the database. Keeping business rules in the Service layer makes the application easier to test, maintain, and modify without affecting persistence logic.

---

1. Explain the JDBC Architecture.
2. What is the role of `DriverManager`?

3. Is `Class.forName()` still required? Why or why not?
4. What is the difference between `Statement`, `PreparedStatement`, and `CallableStatement`?
5. How does `PreparedStatement` prevent SQL Injection?
6. Why is `PreparedStatement` faster than `Statement`?
7. What is `CallableStatement`? When would you use it?
8. What is `ResultSet`? How does its cursor work?
9. Difference between `execute()`, `executeQuery()`, and `executeUpdate()`.
10. What is a JDBC transaction? How do `commit()` and `rollback()` work?
11. What is AutoCommit, and when should it be disabled?
12. What is Batch Processing, and when is it useful?
13. What is Connection Pooling, and why is it important?
14. Explain the DAO pattern with an example.
15. What is the responsibility of the Service layer?
16. Difference between DAO and Service layer.
17. Explain the Factory Pattern with a JDBC example.
18. Why is `DriverManager` considered a Factory?
19. Why is `try-with-resources` preferred in JDBC?
20. How would you optimize JDBC code for performance in a production application?

Topics covered till now: (Java)

Java Fundamentals • JDK, JRE, JVM • Bytecode • Compilation Process • Memory Management • Heap & Stack • Garbage Collection Language Basics • Variables • Data Types • Operators • Type Casting • Scanner Class Conditional Statements • if • if-else • else if ladder • switch case Looping & Control Statement for • while • foreach • break • continue Arrays • 1D Arrays • 2D Arrays ,Classes & Objects • Constructors • Encapsulation • Inheritance • Polymorphism • Abstraction • Association • Aggregation • Composition • Keywords(final,super) Practical Banking Management System Deliverables • UML Diagram • Class Diagram, String Internals • String

Pool • StringBuilder • StringBuffer Other Concepts • Object Class • equals() • hashCode() • toString() • Wrapper Classes • Immutable Objects • Enums • Annotations • Inner Classes • Upcasting • Downcasting, Collections • List • Set • Queue • Map Implementations • ArrayList • LinkedList • HashSet • TreeSet • HashMap • TreeMap Advanced • Comparable • Comparator • Generics • HashMap Internals, • Lambda • Functional Interface • Optional, JDBC Architecture • DriverManager • Connection • Statement • PreparedStatement • CallableStatement Design Patterns • DAO • Service Layer • Factory Pattern, SLF4J • Logger • Logging Standards Logback • Console Logs • File Logs • Rolling Logs Observability Introduction • Why Logging Matters • Production Troubleshooting